Chapter 13

Online Deployment

Chapters 5 to 8 trained a retriever, two rankers, and a value function, and evaluated them offline. This chapter turns those artifacts into services that answer recommendation requests while a user waits: cooperating services with typed contracts between them, a shared deadline, planned fallbacks, and a release process that can undo a bad change. Every pattern is taught in cloud-neutral terms and implemented in companion code that runs on a laptop CPU.

After reading this chapter, you will be able to:

- Trace a recommendation request through a multi-stage serving pipeline and separate the work done ahead of time from the work done within the request deadline.
- Define typed, versioned contracts between services with Pydantic and FastAPI, and decide what a caller does when validation fails.
- Serve the two-tower retriever, the XGBoost lightweight ranker, the MMoE heavy ranker, and the value-function re-ranker from Chapters 5 to 8 as CPU services, and judge when an accelerator is worth adopting.
- Orchestrate the stages under a request deadline with concurrent calls, deadline propagation, bounded retries, circuit breakers, and a ladder of fallbacks.
- Package services as container images, place them on a serverless or Kubernetes runtime, and release new models, indexes, and configurations with health checks, traffic shifting, and rollback.
- Validate a deployed pipeline with manual requests, contract, integration, and smoke tests, and a Locust load test.

13.1 From Notebook to Service

Every model in this book so far has been trained, evaluated, and demonstrated inside a Jupyter notebook or a set of Python scripts. That workflow suits experimentation and produces the trained artifacts this chapter needs, but a model in a notebook cannot answer a user. As a service, the same model must start inside a container, an isolated and packaged runtime environment described in Section 13.9, answer many users concurrently, look up fresh features while someone waits, respond within a latency target, survive a slow or failed downstream service, and keep running for days without leaking memory. This chapter closes that gap for the models from Chapters 5 to 8.

Speed is a product concern as well as an engineering one. Booking.com measured the business cost of latency with a randomized controlled trial in which some users were exposed to synthetic delay, and found that an increase of about 30% in latency cost more than 0.5% in conversion rate (Bernardi et al., 2019). The mitigations the team relied on, including precomputation, caching, batching, and simpler models where they sufficed, appear in this chapter as design choices, and Chapter 14 develops them as optimizations.

13.1.1 Non-Functional Requirements

A non-functional requirement states how well a system must do its job: how fast, how reliably, at what cost, and with how much operational effort. Chapter 2 introduced these requirements for recommender systems (Section 2.2.4), and five of them shape this chapter.

Latency comes first. We describe it with percentiles: the 95th percentile, written p95, is the response time below which 95% of requests complete, and p50 and p99 are defined the same way. A latency target is the response time the system aims to meet at a stated percentile, such as "p95 under 200 milliseconds for the whole request", the example answer in Section 2.2.4, a term Chapter 14 uses throughout. Chapter 15 defines the service level objectives a team commits to and measures over a window of time (Section 15.2.2), and adds objectives that describe recommendation health (Section 15.8.1).

Throughput is the number of requests per second the system sustains. Traffic rises and falls with the time of day, so each stage must scale horizontally: adding more copies of a service, called replicas, should add capacity without code changes. That is easiest when a service is stateless, meaning it keeps no data between requests that changes the next answer, so any replica can serve any request. Every service in this chapter holds only read-only artifacts in memory and is stateless in this sense.

Availability is the fraction of requests that receive a useful answer; a recommendation surface is often on the main path of the product, so each stage needs a planned fallback (Section 13.8). Cost covers compute, memory, storage, and network, paid whether or not traffic arrives. Operability is the effort needed to deploy, observe, and repair the system.

13.1.2 Offline Artifacts and the Online Path

A deployed recommender divides its work into two halves, shown in Figure 13.1. The offline half runs ahead of time on a schedule. Training jobs from Chapters 5 to 8 produce model artifacts, which are the serialized files a service loads, such as an XGBoost `model.json` or a PyTorch checkpoint. A batch job runs the item tower over the catalog and builds an approximate nearest-neighbor (ANN) index, the search structure from Chapter 5 that finds the stored vectors closest to a query vector without comparing against all of them. Feature pipelines from Chapter 4 publish the latest user and item features to an online feature store. Each output carries a version, and a model registry, a versioned store of model artifacts with their metadata, records which versions exist and which one is approved for serving.

The online half runs while a user waits. A client, such as a mobile app or a web page, sends a request that first reaches the network edge. A load balancer spreads incoming connections across the replicas of a service, and it usually performs TLS termination, which means it decrypts the encrypted HTTPS connection so that the services behind it receive plain requests. An API gateway often sits at the same edge and adds authentication, rate limiting, and routing rules. Configuring them is standard web infrastructure and outside the scope of this book.

Behind the edge sits the orchestrator, the service that receives a recommendation request, calls each stage in the right order, enforces the request's deadline, applies fallbacks when a stage fails, and assembles the response. Unlike the gateway, it is application code that understands the recommendation pipeline, and it runs no model itself. The four stage services behind it are retrieval, lightweight ranking, heavy ranking, and the value function with re-ranking, matching Chapters 5 to 8.

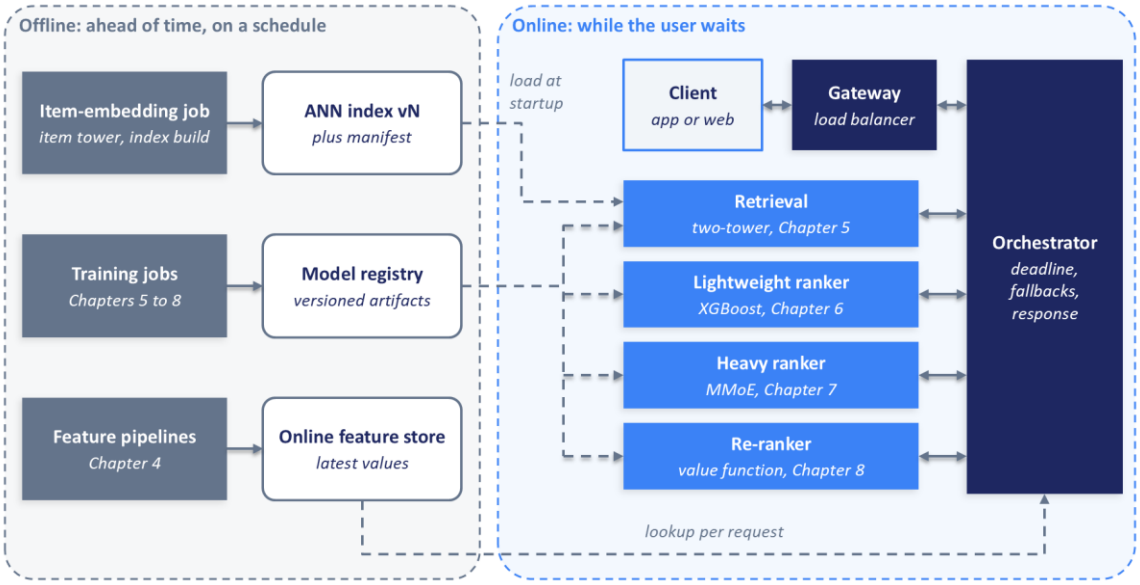


Figure 13.1: Work done ahead of time (left) produces versioned artifacts that the online services (right) load at startup or query while answering a request.

The dashed lines in Figure 13.1 mark the two ways the halves meet. Model weights, encoders, item vectors, and the ANN index are loaded into memory when a service starts, so a request never waits for a file to be read, while feature values change continuously and are looked up per request. The distinction separates failures at startup, where a service can refuse to start, from failures during a request, where the service must answer anyway.

13.1.3 One Request, End to End

Figure 13.2 follows one request through the pipeline. The client sends a user identifier, context such as the time of the request, and the number of results it wants. The gateway forwards the request to an orchestrator replica, which computes the request's deadline, the absolute time by which the response must be sent.

The orchestrator then makes two calls at the same moment, because neither depends on the other. One fetches the user's features from the online feature store. The other asks the retrieval service for candidates; it runs the user tower over the user's recent items to produce a query vector, searches the ANN index, and returns on the order of a thousand candidates with similarity scores (the companion code retrieves 300 from its 20,000-track sample catalog). Item features cannot be fetched in parallel with retrieval, because the orchestrator does not know which items to fetch until retrieval returns, so it fetches them in one batched lookup as soon as the candidates arrive.

The lightweight ranker scores the thousand candidates with the XGBoost model from Chapter 6 and keeps roughly the best hundred. The heavy ranker scores those with the MMoE model from Chapter 7 and returns five predictions per item. The value function and re-ranker from Chapter 8 combine the predictions into one value per item, diversify the list with Maximal Marginal Relevance, and apply business rules such as artist pacing. The orchestrator adds display metadata for the final items and returns the response. In a music service the metadata would be the title, artist name, and artwork; in the companion code, which uses Yambda's anonymized catalog, it is the artist and album identifiers and the track length.

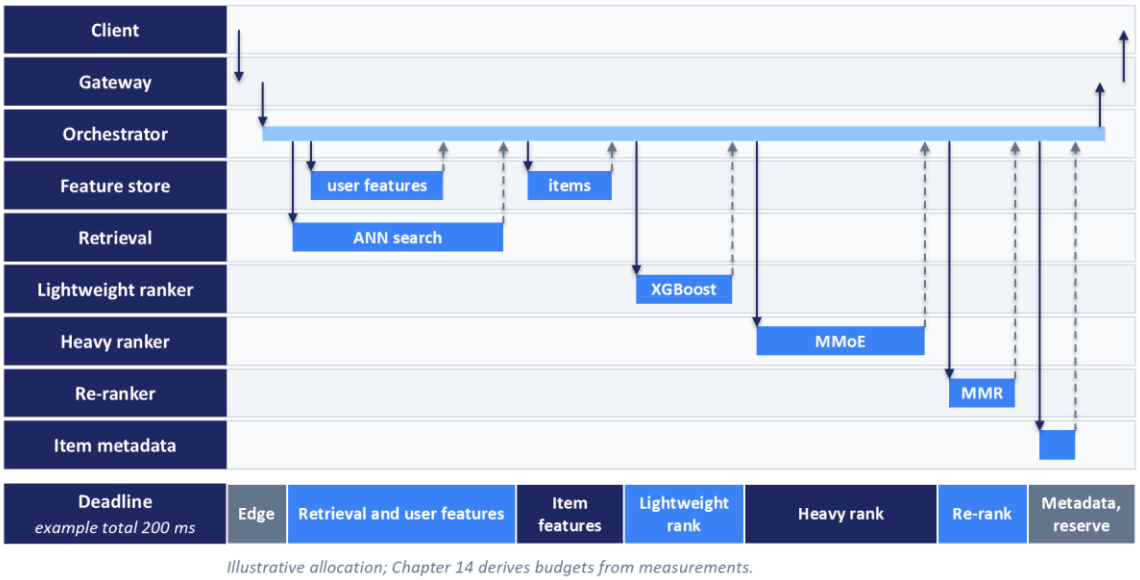


Figure 13.2: One request under a deadline. User features are fetched in parallel with retrieval; item features follow in one batched lookup. The bar at the bottom shows an illustrative split of the deadline across stages, which Chapter 14 replaces with budgets derived from measurement.

The bar at the bottom of Figure 13.2 divides an example latency target of 200 milliseconds among the stages, with the heavy ranker and retrieval receiving the largest shares and the orchestrator keeping a reserve for network hops, serialization, and metadata. The allocation is only a planning tool, because percentiles do not add. Section 14.2.3 shows why, and Section 14.9.1 derives stage budgets from measured latencies; Section 13.8 turns the deadline into per-call timeouts.

13.1.4 What We Deploy

The deployment draws directly on the artifacts trained in Chapters 5 to 8. Table 13.1 lists them and the service that loads each one.

Table 13.1: Model artifacts from Chapters 5 to 8 and the services that load them.

Source chapter	Artifact	Loaded by
Chapter 5 (retrieval)	Two-tower model in PyTorch: a user tower with attention pooling or a Transformer encoder, and an item tower, both producing 50-dimensional vectors; retrained on Yambda for this chapter, with Yambda's 128-dimensional audio embeddings as content embeddings	Retrieval service
Chapter 6 (ranking)	XGBoost binary classifier (`model.json`) and its fitted label and frequency encoders (`feature_encoder.pkl`); label: the user listens to at least 50% of the track; trained on Yandex Yambda	Lightweight ranker

Source chapter	Artifact	Loaded by
Chapter 7 (advanced ranking)	MMoE multi-task model (`mmoe.pt`; Chapter 7's training saves it as `best_model.pt`) and its feature processor (`feature_processor.pkl`); five heads: engagement, completion, play_ratio, like, and dislike; trained on Yambda	Heavy ranker
Chapter 8 (value functions)	Value-function weights, the MMR diversity parameter, and business rules; configuration only, with no trained weights	Value function and re-ranker

The retrieval model is retrained on Yambda for this chapter so that every stage shares one catalog, and the candidates retrieval returns are the tracks the rankers score.

The companion code implements the pipeline as five FastAPI services. FastAPI is a Python web framework in which you write ordinary Python functions, declare the types of their inputs and outputs, and get HTTP endpoints that parse and validate each request (FastAPI, 2026). Each service runs under Uvicorn, a lightweight Python web server (Uvicorn, 2026). The required path runs all five services on one machine, on CPU, with no cloud account and an in-process FAISS index. Section 13.11 describes its tests and load test, and the README lists the exact commands.

13.2 Contracts Between Stages

In a pipeline whose services are developed, deployed, and scaled independently, many failures happen at the boundaries. One team renames a response field from `item_id` to `product_id`; another adds a required request field without telling its callers; a feature is renamed in the feature store while the model still expects the old name. These mistakes at least produce errors. Harder to see are changes that keep the shape of the data and alter its meaning, such as a duration that switches from seconds to milliseconds, which can degrade recommendations for weeks before anyone notices.

A contract is the explicit agreement about the requests a service accepts and the responses it returns: which fields exist, what type each has, which are required, and what values are allowed. In Python the usual tool for writing contracts is Pydantic, a library that validates data against schemas declared as Python classes with type annotations (Pydantic, 2026). FastAPI builds on it: when a request arrives, FastAPI parses the body into the declared class, and Pydantic checks required fields, types, and constraints such as minimum and maximum values. If validation fails, FastAPI rejects the request with HTTP status 422, Unprocessable Content, and a body that lists each failing field and the reason, before any of the service's own code runs (FastAPI, 2026).

Plain Python dictionaries offer none of this, and a model fed malformed input returns confident, wrong output without raising an exception. Pydantic's default lax mode converts compatible values, such as the string `"1"` to the integer `1`, while its strict mode rejects them (Pydantic, 2026).

The companion code defines every contract in one shared Python package that each service imports, validated at runtime on every request and checked by the test suite, which compares them with a stored snapshot of the published schemas, so a schema change that breaks a caller fails a test before it reaches a running service.

```
from pydantic import BaseModel, Field

class ScoredItem(BaseModel):
    item_id: str
    score: float
    task_scores: dict[str, float] | None = None  # set by the heavy ranker only

class RankingRequest(BaseModel):
    trace_id: str
    user_id: str
    candidate_ids: list[str] = Field(min_length=1)
    user_features: dict[str, float | int | str | None]
    item_features: dict[str, dict[str, float | int | str | None]]
    top_k: int = Field(gt=0)

class RankingResponse(BaseModel):
    trace_id: str
    model_version: str
    items: list[ScoredItem]
```

The sketch simplifies the ranking contract. The companion code uses integer identifiers, sends each candidate as an object carrying its own features, names the size `num_results`, and carries the trace identifier and the deadline in HTTP headers; every response also repeats the trace identifier in its body. The contracts package holds the full definitions. A ranking service declares its endpoint with `@app.post("/v1/rank", response_model=RankingResponse)` and a function that takes a `RankingRequest`, and FastAPI validates the incoming body and the outgoing result, so a bug that drops a field from the response is caught inside the service that caused it.

13.2.1 The Contract Chain

The output of one stage must be valid input for the next, so the contracts form a chain of four request and response pairs, shown in Figure 13.3.

The client pair is the external API. The client sends a user identifier, a context object with the request time and an organic flag that says whether the listen is user-initiated rather than recommended, and the number of results it wants; the response returns a ranked list with an identifier, a score, and the artist, album, and track length of each item.

The retrieval request carries the user identifier, the number of candidates wanted, and any eligibility filters, which are rules that exclude items from consideration, such as items the user has already played today. The response returns candidates with their similarity scores, the version of the index that produced them, and a flag saying whether they came from the ANN search or from the popularity fallback.

The ranking pair is shared by the lightweight and heavy rankers. The request carries the candidate identifiers, the user and item features fetched by the orchestrator, the request's context, and the number of items to keep. The response returns scored items, and the heavy ranker's response adds five task scores per item, keyed `engagement`, `completion`, `play_ratio`, `like`, and `dislike`.

The re-rank request carries the heavy ranker's scored items, the number of results wanted, and optionally a policy version, which lets an experiment pin a particular value-function configuration. The

response returns the final ordered item identifiers with their values and the configuration version that produced them.

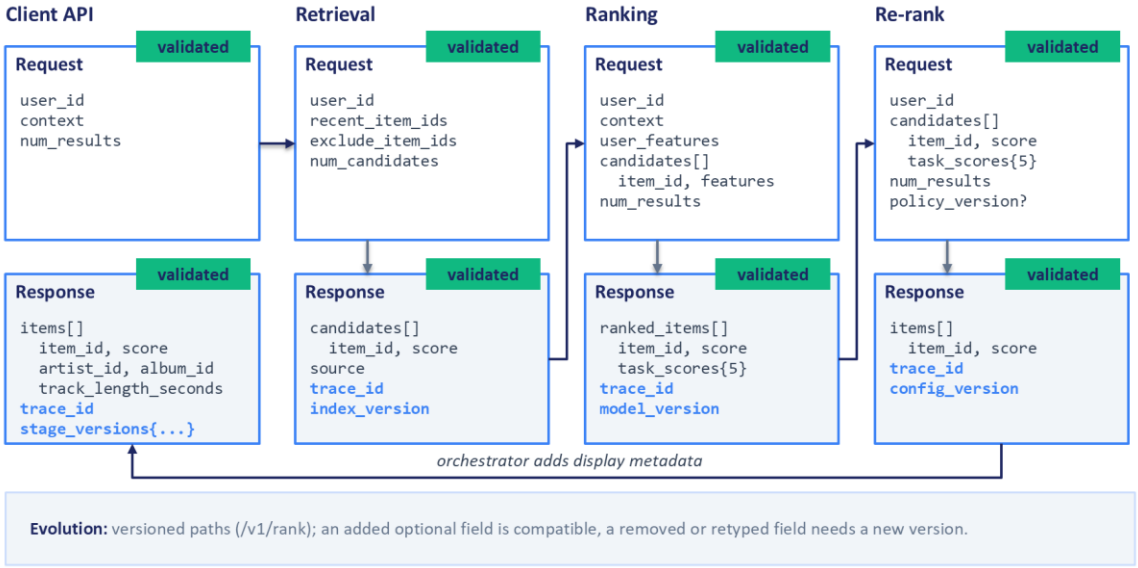


Figure 13.3: The contract chain. Each response feeds the next request, every boundary is validated, and every response carries the trace identifier and the version of the artifact that produced it.

The re-rank response is deliberately different from the client response. The ranking stages work only with identifiers and scores, which keeps their payloads small, and the orchestrator looks up display fields from an item metadata store (in the companion code, the feature store's item table), usually through a cache, only for the final ten or twenty items, far cheaper than carrying them through a thousand candidates.

13.2.2 When Validation Fails

What a service does with a rejected request depends on where the rejection happens. If the client's request to the orchestrator fails validation, the client sent something malformed, and the orchestrator returns the 422 response to the client unchanged; retrying cannot fix a client bug.

If one of the orchestrator's own calls to a stage is rejected, two services that the same organization deploys disagree about a contract, most often because one of them was just released. The orchestrator should not retry, because errors caused by the request itself recur on every attempt (Brooker, 2019). It should log the rejection with the field-level detail, the trace identifier, and the versions of both services, then continue with that stage's fallback from the degradation ladder in Section 13.8. Rejections should be counted per stage, with an alert above a small threshold, because a burst of them right after a release is a strong signal to roll that release back (Section 13.10). Validation runs in both directions: a stage response that fails validation, such as a heavy-ranker reply with missing task scores, counts as a failure of that stage and triggers the same fallback.

13.2.3 Evolving a Contract

Contracts change as models and clients evolve. Each service in this chapter exposes its endpoints under a version prefix, such as `/v1/retrieve` or `/v1/rank`, a scheme called URI versioning. Adding an optional field with a default value is a compatible change. Removing a field, renaming it, or changing its

type is a breaking change and requires a new version, such as `/v2/rank`, served alongside the old one until every caller has moved.

Old and new code run side by side during every release, so a gradual release needs both backward compatibility, where newer code can read data written by older code, and forward compatibility, where older code can read data written by newer code (Kleppmann, 2017). Pydantic helps with forward compatibility, since by default it ignores fields it does not recognize. A breaking change is safest as a sequence: add the new field while keeping the old one, move every caller to it, and remove the old one in a later release.

13.2.4 The Feature Contract

One more contract governs the features the rankers consume: which features exist, what they are called, how they are encoded, what default applies when a value is missing, and how old a value may be. The XGBoost model from Chapter 6 was trained on features such as `track_length_seconds`, `hour_of_day`, and `is_organic`, with specific encodings. If the feature store serves a feature under a different name or encoding, the model receives meaningless input and still returns scores, the training-serving skew that Chapter 4 described (Section 4.3).

A practical form for the feature contract is a configuration file that lists, for every feature, its name, type, encoding, default, and maximum staleness. At startup, a ranker verifies that every feature the model expects is declared with a matching type and encoding, and refuses to report itself ready if not, so it never receives traffic (Section 13.10.2). At request time, the feature store client treats any value older than its maximum staleness as missing, and the contract's declared default, or a missing marker the model is known to handle, replaces it before the ranker sees the request. Each substitution is counted, because a rising rate of defaults is one of the earliest signs of a broken feature pipeline, a point Chapter 15 develops into a monitored objective.

13.2.5 Trace and Version Fields

Every request in the pipeline carries a trace identifier, a unique value created by the orchestrator, or by the gateway if it already assigns one, and passed to every downstream call. The idea comes from distributed tracing: Google's Dapper system tags all the work done for one request with a shared trace identifier so the full tree of calls can be reconstructed (Sigelman et al., 2010). This chapter uses the simplest form of the idea, in which every service writes the trace identifier into its request log lines and each response.

Every response also states the version of the artifact that produced it: the model version from each ranker, the index version from retrieval, and the configuration version from the re-ranker. With the trace identifier, these fields answer the first questions asked during an incident, which versions served this request and what each stage returned, and Chapter 15 relies on them to localize a regression to one stage.

13.3 Retrieval

The two-tower model from Chapter 5 is well suited to deployment because its two towers do not interact until the final dot product. The item tower can therefore run offline in a batch job that computes a vector for every item in the catalog, while the user tower runs online and turns the user's recent activity into a query vector for each request. An approximate nearest-neighbor search then finds the items whose vectors score highest against the query. Instagram's Explore system follows this

pattern: its item tower generates embeddings in an offline pipeline that are stored in an ANN index, while its user tower runs on the fly with the freshest user features (Vorotilov and Shugaepov, 2023). YouTube's candidate generator was likewise served as a nearest-neighbor search over precomputed video vectors (Covington et al., 2016).

The split makes dense retrieval affordable. If both towers had to run for every user and item pair, a catalog of a million items would cost a million forward passes per request; with the split, each request costs one forward pass of the user tower and an index search.

13.3.1 Two Embedding Tables

A retrieval service built on the Chapter 5 model holds two different sets of item vectors, and confusing them is an easy mistake. The first is the content-embedding table, which maps each item identifier to the input representation the user tower consumes; the Chapter 5 user tower reads a sequence of these vectors, one per recently played track, and pools them with attention or a Transformer encoder into a single query vector. The second set is the item vectors, the outputs of the item tower, which live in the ANN index and define the space the query is searched against.

Both sets are keyed by item identifier. In the companion code the content embeddings are Yambda's 128-dimensional audio embeddings and the item vectors are 50-dimensional, so a mix-up fails on a dimension mismatch; where the two sizes coincide, the same mistake runs without error and returns well-formed, meaningless candidates. Give the two tables distinct names in code.

13.3.2 Building the Item Index Offline

The offline job loads the trained two-tower checkpoint, runs the item tower over the whole catalog in batches, and collects the outputs into a matrix with one row per item; computing it is a single batched pass on a CPU. The job then builds an ANN index over the matrix, writes the index, the mapping from index positions to item identifiers, and the content-embedding table to versioned storage, and writes a manifest that names all of them.

The job runs on a schedule, daily or more often when the catalog changes quickly, and it must run again whenever the two-tower model is retrained, because a new model defines a new vector space and every item vector changes.

13.3.3 Choosing Where the Index Lives

In Chapter 5 we used FAISS, a library for efficient similarity search over dense vectors (Johnson et al., 2019). FAISS runs inside the application process: each retrieval replica loads its own copy of the index into memory and searches it with a function call, with no network hop. That is the right choice for the companion code and for catalogs whose index fits comfortably in a replica's memory. FAISS also supports some maintenance: indexes that store explicit identifiers accept `add_with_ids`, and several index types support `remove_ids`, usually at the cost of a pass over the index (Faiss, 2026a), though graph-based HNSW indexes do not support removal (Faiss, 2026b). A search can also be restricted to a subset of identifiers through an `IDSelector` (Faiss, 2026c). FAISS does not provide filtering on item attributes stored alongside the vectors, replication between replicas, or a network service; if you need those, you build them.

The alternatives are a vector database that your team runs as a separate service, such as Qdrant, which adds updates and deletions over an API, filtering on stored attributes, and sharding and replication across nodes (Qdrant, 2026), or a managed vector search service from a cloud provider or vendor, which

takes over operations in exchange for a network hop and less control. Table 13.2 compares the three categories. Product names for managed services appear in Table 13.6, because they change more often than the categories do.

Table 13.2: Options for hosting the ANN index.

Consideration	In-process library (FAISS)	Self-hosted vector database (for example, Qdrant)	Managed vector search service
Where it runs	Inside each retrieval replica	Separate service operated by your team	Separate service operated by a provider
Network hop per query	None	Yes	Yes
Adding and removing items	Add with identifiers; removal for some index types, often a full pass; none for HNSW	Upserts and deletions through the API	Upserts and deletions through the API, with provider-specific limits
Filtering	Restrict search to a set of identifiers	Filters on stored attributes	Filters on stored attributes, with provider-specific syntax
Replication and scaling	Each replica holds a full copy; scale by adding replicas	Sharding and replication that you configure	Handled by the provider
Operational burden	Lowest; the index ships as an artifact	Highest; you run a stateful cluster	Low; you manage configuration and cost
Fits	Indexes that fit in one replica's memory; the companion code	Large or fast-changing catalogs with attribute filtering	Teams that prefer to buy operations

13.3.4 Keeping the Tower and the Index in Step

The user tower and the index must come from the same model version. The user tower produces query vectors in the space defined by the model that trained it, and the index answers in the space of the model that produced its item vectors. When the versions diverge, for example because a new user tower was deployed before the matching index finished building, every search still returns results, and those results are close to random.

The defense is to treat the user tower, the content-embedding table, the item vectors, and the index as one unit with one version. The offline job writes a manifest, a small file that names the model version and the exact files of each artifact in the unit. The retrieval service loads a manifest at startup, loads every artifact it names, checks that they agree, and refuses to report itself ready if they do not. To move to a new version, the service loads the complete new unit alongside the old one and then switches a

single in-memory reference from the old unit to the new one, so that no request ever sees a new tower paired with an old index. Section 13.10 generalizes this to a version pointer for any model or index, and Chapter 15 covers the release checks and compatibility windows that decide when a new unit may be switched in (Section 15.9.4).

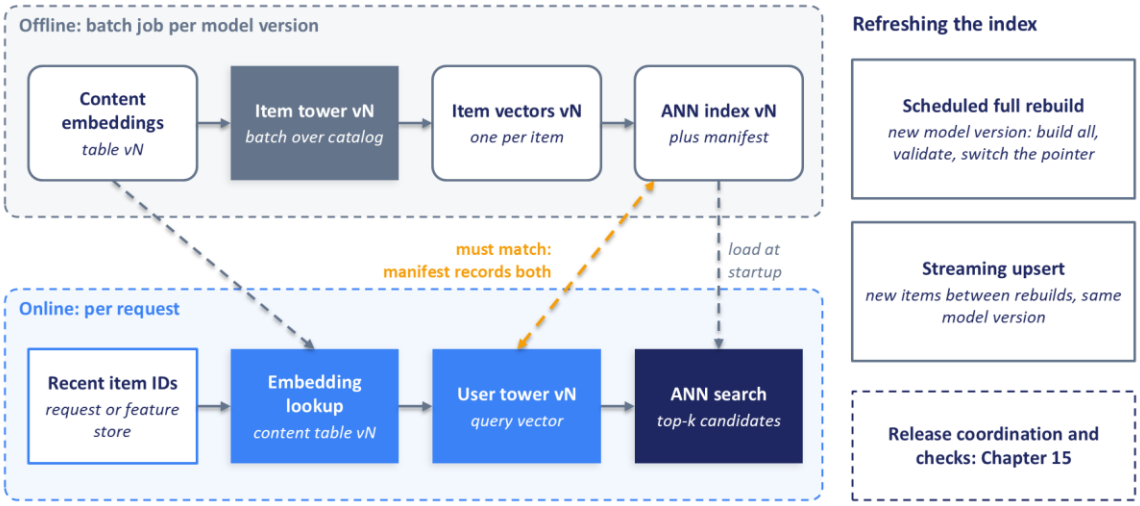


Figure 13.4: The two-tower deployment split. The item tower runs offline to build a versioned index; the user tower runs per request. The user tower and the index must come from the same model version, which the manifest records.

The catalog changes between model versions as well, and Figure 13.4 shows the two common ways to keep the index current. A scheduled full rebuild recomputes every item vector and builds a fresh index, nightly for example; it is easy to reason about, but a new item stays invisible until the next rebuild. Where freshness matters, as with news or limited-time offers, a streaming path computes each new item's vector with the serving item tower as the item arrives and adds it to the live index. Periodic full rebuilds remain necessary, both when the model changes and because clustered index types such as IVF keep the cluster centers learned at build time, which fit a changing catalog less well as additions accumulate.

13.3.5 The Online Retrieval Service

The retrieval service is a FastAPI application with one endpoint, `POST /v1/retrieve`. At startup it reads the manifest and loads the user tower weights, the content-embedding table, the FAISS index with its identifier mapping, and the popularity list described below, all of which stay in memory for the life of the process.

For each request, the service obtains the user's recent item identifiers, from the request if the caller supplied them or from the online feature store otherwise, looks up their content embeddings, pads or truncates the sequence to the length the user tower was trained with, and runs one forward pass on CPU to produce the query vector. It then searches the index and returns the top candidates with their scores and the index version.

Before returning candidates, the service applies eligibility filtering, which removes items that must never be recommended in this context: items that are unavailable or withdrawn, blocked in the user's region, or already consumed when the surface calls for new ones. Pre-filtering restricts the ANN search itself to eligible identifiers, which FAISS supports through an `IDSelector` and vector databases through

attribute filters. Post-filtering searches for more candidates than needed, say twice as many, and drops the ineligible ones afterward; it is simpler, but a rule that excludes a large share of the catalog can leave too few candidates, so the service should count how often it returns fewer than requested. A production re-ranker applies the hard business rules again as a final safeguard (Section 13.6.4); in the companion code, retrieval applies the exclusions inside the search and the re-ranker applies the artist and album caps.

13.3.6 Users Without History

A user with no history gives the user tower nothing to encode. This is the user cold-start problem, which is unrelated to the container cold start discussed in Section 13.5, where a newly started replica is slow to answer its first requests. The standard fallback for a user without history is popularity-based retrieval, introduced in Chapter 5 (Section 5.2.1): return the most popular items overall, or within a segment inferred from context such as the region or the device.

If the user's history is empty, the retrieval service skips the user tower and returns candidates from a popularity list that a batch job computes and the service loads at startup, marked as coming from the popularity source. The same list serves as the orchestrator's fallback when the retrieval service fails entirely (Section 13.8). Because the fallback keeps the user experience intact, it can also hide a broken retriever, so the service counts how often each source is used, a rate Chapter 15 monitors.

13.4 Lightweight Ranking

The lightweight ranker narrows roughly a thousand retrieved candidates to the hundred or so that deserve the heavy ranker's attention. Its job is to be cheap per candidate while discarding few good items. Gradient-boosted decision trees such as XGBoost are a common choice for this stage, because they score quickly on a CPU, work well with the tabular features available at this point, and tolerate missing values (Chen and Guestrin, 2016). They are not the only choice: Instagram's Explore system uses a lightweight neural first-stage ranker, a two-tower model trained to approximate its second-stage ranker (Vorotilov and Shugaepov, 2023). Either kind of model deploys the same way, as a stateless CPU service that scores a batch of candidates per request. This chapter serves the XGBoost model from Chapter 6.

13.4.1 Loading the Model and Its Encoder

The Chapter 6 model is a binary classifier that predicts whether the user will listen to at least 50% of a track. It was saved as two artifacts. The file `model.json` holds the trees in XGBoost's native format, and `feature_encoder.pkl` holds the fitted label and frequency encoders that turn raw feature values into the numbers the model was trained on. The service loads both once at startup, the model through Chapter 6's `XGBoostRanker.from_pretrained`, which loads it into XGBoost's scikit-learn `XGBClassifier`, and the encoders through Python's `pickle` module, and keeps them in memory for the life of the process.

Two cautions apply to the pickled encoder. Unpickling a file can execute arbitrary code, so the service must load encoders only from the trusted artifact store that the release process writes to, never from a path supplied in a request. A pickled object also depends on the library versions that created it, so the service's container image should pin the versions used in training, and a startup check that encodes a known sample and compares the result with a stored expected value catches a mismatch before the service takes traffic; the companion code runs this comparison in its tests.

13.4.2 Assembling Features for a Batch of Candidates

The request carries the request's context, the user features that the orchestrator fetched in parallel with retrieval, and the item features for every candidate, which it fetched in one batched lookup once the candidates were known. Some features are derived at request time instead of being stored. The hour of the day, for example, comes from the request timestamp, and it must be computed by the same function that computed it for training, imported from shared code, because a second implementation is a common source of training-serving skew. The service then passes every feature through the fitted encoders. Categories never seen in training, such as a newly released artist, need a defined mapping, typically to an "unknown" code, and the encoder must provide it.

When a feature value is missing because it was never computed or because the feature store client discarded it as stale (Section 13.2.4), the feature store client substitutes the contract's default before the ranker sees the request, as in the companion code. A model trained with missing values can instead receive a missing marker, and XGBoost then follows the default direction each split learned for missing values (Chen and Guestrin, 2016). Either way, a partial outage of one feature degrades scores instead of failing requests, but a feature that silently goes missing for every request also produces plausible scores and no errors. Substitutions should be counted per feature, as the companion code's feature store client does. Chapter 15 opens with the harder case, in which an upstream default replaces missing values with zeros that no missing-value count can see.

13.4.3 Scoring and Trimming

With the features encoded, the service builds one feature matrix with a row per candidate and calls XGBoost's predict function once for the whole matrix. One call for a thousand rows is much faster than a thousand calls for one row each, mainly because the fixed cost of each call, building the input structure and crossing from Python into XGBoost's native code, is paid once instead of a thousand times. Chapter 14 measures this effect and the other batching opportunities in the pipeline.

The service sorts the candidates by score, keeps the requested number, and returns them with the model version. XGBoost can also predict on a GPU, which pays off for very large batches. For a thousand rows per request, a CPU is the natural starting point, and the lightweight ranker's container needs only Python, XGBoost, its encoders, and the web framework.

13.5 Heavy Ranking

The heavy ranker scores the hundred or so survivors of lightweight ranking with the Multi-gate Mixture-of-Experts (MMoE) model from Chapter 7 (Ma et al., 2018). The model is a neural network with shared expert sub-networks, a gating network per task that decides how much each expert contributes, and a small output tower per task. In Chapter 7 it predicts five targets from the Yambda data: engagement, completion, play ratio, like, and dislike. Sparse features such as the user and item identifiers enter through learned embedding tables that map each identifier to a 16-dimensional vector, trained jointly with the rest of the network.

13.5.1 What Makes the Heavy Ranker Different

The heavy ranker differs from the lightweight ranker in three ways that affect deployment: every candidate passes through several experts and five towers, so its forward pass costs more per candidate; its embedding tables grow with the number of users and items, which can make memory the binding constraint at scale; and it produces several outputs per candidate, which the value function depends on.

The service loads the Chapter 7 artifacts at startup: the weights in `mmoe.pt`, a checkpoint that also records the model and feature-set versions, and the fitted feature processor in `feature_processor.pkl`. It builds the model from the same class definition used in training, loads the weights with strict key matching so that any mismatch between code and checkpoint stops the service at startup, and switches to evaluation mode with gradient tracking disabled. Each request is scored in one forward pass over all its candidates, which is batching within a request, distinct from the batching across requests discussed below.

Some teams export the model before serving it, to TorchScript, PyTorch's serialized program format; to ONNX, an open, framework-neutral format; or through TensorRT, NVIDIA's compiler for its GPUs, which run general computation through NVIDIA's CUDA platform. Section 14.8.3 covers export; the companion code serves the model directly with PyTorch on CPU, which keeps the training and serving code identical.

13.5.2 CPU First, Accelerators After Measurement

The book's MMoE is small. With 16-dimensional embeddings, experts of sizes 256 and 128, and towers of sizes 64 and 32, a forward pass over a hundred candidates amounts to tens of millions of multiply-add operations, a modest workload for a modern CPU. The default for this model is therefore a CPU service, and the companion code serves it that way. Its README explains how to measure the heavy ranker's latency on your own machine.

An accelerator such as a graphics processing unit (GPU) becomes worth its cost and operational overhead when measurement shows one of four conditions: compute per request, where a deeper model, long behavior sequences, or many more candidates push CPU latency past the stage's share of the deadline; throughput per replica, where the CPU replicas needed at peak cost more than fewer accelerator replicas; batch size, because an accelerator is efficient only on large batches and may need batching across requests, which adds waiting time; and embedding memory, where tables outgrow one machine and need sharding (Section 13.5.3), which an accelerator does not solve by itself. The comparison is cost per thousand requests at the latency target, measured under load (Section 13.11 and Chapter 14). We name no specific accelerator models, because the options and their price-performance change faster than a book stays in print.

Accelerators also change the cold-start picture. A container cold start is the delay before a newly started replica can answer: the platform starts the container, and the process imports its libraries, loads the model weights, and warms up. Some platforms offer scale-to-zero, removing every replica when traffic stops and starting one when a request arrives, so an idle service costs almost nothing. Cloud Run, Google Cloud's serverless platform for containers, can scale GPU-backed services to zero, and its documentation reports that a GPU instance with drivers installed starts within seconds (Google Cloud, 2026a). Loading the model comes on top of that, and the first request after an idle period waits for all of it, which is incompatible with a latency target in hundreds of milliseconds. Services on the request path therefore keep a minimum of one warm replica, which the platform bills even when idle (Google Cloud, 2026a); the provider's pricing page gives the current rates (Google Cloud, 2026e). The same trade-off applies on every provider and to CPU services, with smaller costs on both sides.

Scaling decisions need a signal. The Horizontal Pod Autoscaler of Kubernetes, a system for running containers across a cluster of machines (Section 13.9.3), reads CPU and memory use through the cluster's standard metrics API, and any other metric, such as requests in flight, requires an adapter that publishes it through the custom metrics API (Kubernetes, 2026c). GPU utilization is a poor signal for inference: Google's guidance, written for serving large language models on Kubernetes, notes that it

measures whether the GPU is busy, not how much work it is doing, and recommends scaling on server metrics such as queue size or batch size (Google Cloud, 2026c). For this chapter's services, the natural signals are concurrent requests per replica and request latency, which serverless platforms such as Cloud Run use directly.

13.5.3 Sharded Embedding Lookup

As recommendation models grow, the embedding tables often become their largest component. With hundreds of millions of users and tens of millions of items, the tables can exceed the memory of one machine even when the dense network is small. Facebook's DLRM work describes this pressure and handles it by splitting the embedding tables across devices while replicating the smaller dense layers (Naumov et al., 2019).

At serving time the usual answer is a sharded embedding lookup service, shown in Figure 13.5. The tables are split into shards by a rule such as the hash of the identifier modulo the number of shards, and each shard runs on its own machine. A router sends each identifier's lookup to the shard that holds it, all shards answer in parallel, and the ranker concatenates the returned vectors and runs the dense part of the model. The pattern descends from the parameter server of distributed training, in which server nodes hold the shared parameters while workers compute updates against them (Li et al., 2014); the serving version is simpler, because its shards are read-only between releases. Libraries such as TorchRec provide sharding strategies (by table, row, or column), a planner that chooses a placement, and inference support for sharded tables (Ivchenko et al., 2022).

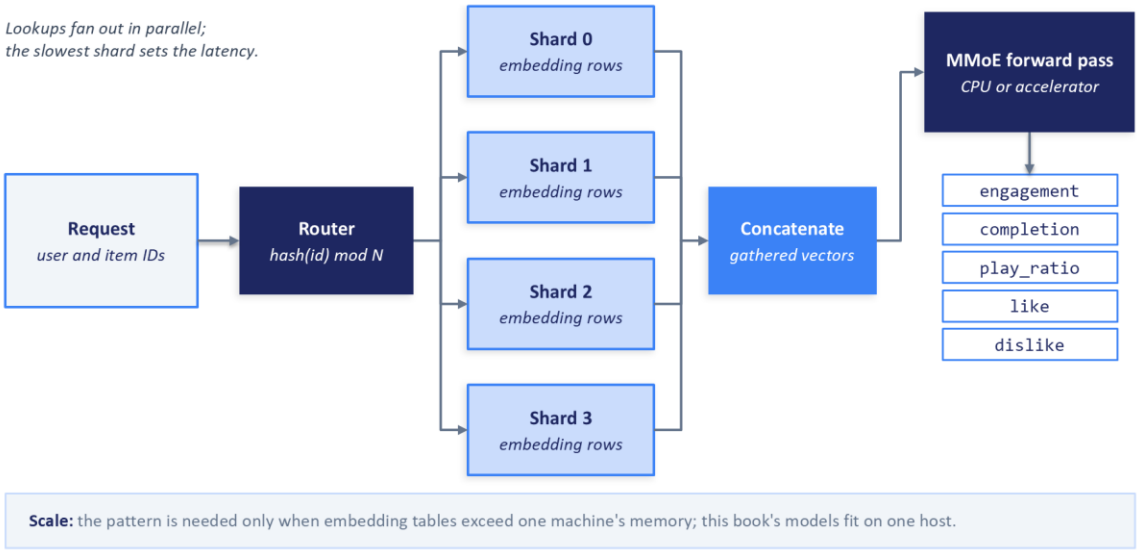


Figure 13.5: Sharded embedding lookup. Identifiers are routed to the shard that holds them, all shards answer in parallel, and the gathered vectors feed the dense network. The pattern is needed only when the tables exceed one machine's memory.

Sharding has a latency cost: the lookup finishes only when the slowest shard answers, so the stage's tail latency grows with the number of shards; Section 14.5.2 works out how fan-out raises tail latency. The Yambda-scale tables in this book fit comfortably in one process's memory, so the companion code does not shard.

13.5.4 Serving Frameworks

A model server is a program dedicated to running trained models behind a network API. Table 13.3 compares the custom web service used in the companion code with two maintained options. Dynamo-Triton, formerly NVIDIA Triton Inference Server, runs models from several frameworks on GPUs and CPUs, with dynamic batching, concurrent model execution, and built-in metrics (NVIDIA, 2026). Ray Serve is a Python library for composing several models into one application, with batching and autoscaling, and it integrates with FastAPI (Ray, 2026). Dynamic batching means the server holds incoming requests for a few milliseconds and runs them through the model as one batch, trading a small wait for higher throughput; Section 14.8.2 explains when batching pays off, and Exercise 2 Part B (Section 14.13) measures it.

TorchServe is now in limited maintenance, with no planned updates or security patches (PyTorch, 2026). In the table, Prometheus is an open-source monitoring system that collects numeric metrics by periodically reading an HTTP endpoint on each service.

Table 13.3: Options for serving the heavy ranker.

Dimension	Custom web service (FastAPI and PyTorch)	Dynamo-Triton (formerly Triton Inference Server)	Ray Serve
What you write	The endpoint, model loading, and batching logic	A model repository with a configuration file per model	Python classes for each deployment, with optional FastAPI routes
Model formats	Whatever your Python code can load	PyTorch, ONNX, TensorRT, tree models, and Python backends	Any Python model
Batching across requests	You implement it	Built in, configurable	Built in, configurable
Several models per process	You implement it	Built in	Built in, with model composition
Hardware	CPU or GPU through PyTorch	GPUs and CPUs	CPUs and GPUs on a Ray cluster
Metrics and health checks	You implement them	Built in, with Prometheus integration	Provided through Ray's monitoring tools
Fits	Small models, one model per service, teams that want minimal moving parts; the companion code	High-throughput accelerator serving and many models per host	Python teams composing several models with autoscaling

13.5.5 The Multi-Task Output Contract

The MMoE returns five outputs per candidate in one forward pass, and the response must present them in the form the value function expects. Four heads are binary classifiers whose raw outputs are logits, so the service applies a sigmoid to turn each into a probability. The play-ratio head is a regression output: it estimates the expected fraction of the track the user will play, and it is returned as that estimate with no sigmoid applied unless the model was trained with one. If Chapter 8's calibration step was fitted for this model, the calibration mapping is part of the model artifact and ships under the same version.

The heavy ranker's response therefore contains, for each item, its identifier, a default score equal to the completion probability, the quantity the lightweight ranker also estimates, which the orchestrator uses as a sort key if the re-ranker is unavailable, and a task-score dictionary with the keys `engagement`, `completion`, `play_ratio`, `like`, and `dislike`. In Chapter 8's value function, the engagement head supplies $P(\text{listen})$, the like and dislike heads supply $P(\text{like} \mid \text{listen})$ and $P(\text{dislike} \mid \text{listen})$, and the play-ratio head supplies $E[\text{play_ratio}]$; the completion head supplies the default score and is available to policies that use it. When task scores are present, the re-ranker's contract requires all five keys, so a model release that renames or drops a head fails validation at the boundary instead of silently zeroing a term in the value function. The field is absent only on the fallback path in Section 13.8.5.

13.6 Value Function and Re-ranking

The value function and re-ranker from Chapter 8 contain no trained weights. The value function is a formula with configurable coefficients, the Maximal Marginal Relevance (MMR) re-ranker is an algorithm with one tunable parameter, and the business rules are constraints written in a configuration file. That makes this the fastest stage to change: a new ranking policy needs a new configuration, with no retraining and no new container image. It also means that a configuration change alters what every user sees, so it deserves the same care as a model release.

13.6.1 Why This Is a Separate Service

It is tempting to fold the value function into the heavy ranker, since it only applies a formula to the MMoE outputs. We keep it separate for two reasons, at the cost of one extra network call per request. The first is ownership: the heavy ranker's team works on model quality, while the ranking policy, meaning how much a like is worth relative to a listen, belongs to product owners, and the two change on different schedules.

The second reason is experimentation. Each value-function configuration is a different ranking policy, and keeping the value function in its own service lets a team run A/B tests that compare policies without touching the model (Chapter 16 covers how to design and analyze those tests). It also lets the business adjust weights for a period, such as raising the weight on likes during a promotion. Instagram's Explore system has a comparable final step, a value model that combines predicted probabilities for several events with weights before the final ordering (Vorotilov and Shugaepov, 2023).

13.6.2 Policy as Configuration

The value function from Chapter 8 combines the heavy ranker's predictions into one number per item:

$$\text{Value} = w_{\text{listen}} P(\text{listen}) + w_{\text{like}} P(\text{listen}) P(\text{like} \mid \text{listen}) + w_{\text{engagement}} P(\text{listen}) E[\text{play_ratio}] - w_{\text{dislike}} P(\text{listen}) P(\text{dislike} \mid \text{listen})$$

The weights, the MMR parameter λ, and the business rules live in one configuration file with a version identifier. The service loads the active version at startup and checks periodically for a newer one, replacing it in memory without a restart. Hot reloading lets a policy change reach production within minutes, and it is safe only with the safeguards below.

Validation comes first. Every configuration is parsed into a Pydantic model before it can become active, so a weight that is negative or missing, a λ outside the range from 0 to 1, or an unknown rule type is rejected, and the service keeps the current version and reports the rejection in its log and health check, where an alert can pick it up. Running the same validation in CI, when someone proposes the change, catches most mistakes before they reach a service.

Versions are immutable. Each change is published as a new file under a new version identifier, with its author and reason recorded, and a small pointer names the version that should be active. This gives an audit trail of every policy that has served traffic, which Chapter 15 relies on when a quality metric moves and the first question is what changed. Sculley et al. (2015) count configuration as technical debt: large systems accumulate many configuration options, their testing is often treated as unimportant, and a configuration mistake can be as costly as a code bug.

Rollout is staged: a new configuration first serves a small share of traffic and reaches everyone only after its metrics look healthy, using the traffic-shifting mechanics in Section 13.10. Reverting is fast: moving the pointer back restores the old policy at each replica's next poll. Replica skew is expected, because replicas check for new versions independently and for up to one polling interval serve different versions. Every re-rank response therefore names the configuration version that produced it, and an experiment that needs a stable assignment can pin a version through the request's policy-version field (Section 13.2.1).

13.6.3 MMR Diversity Re-ranking

After computing values, the re-ranker diversifies the list with MMR (Carbonell and Goldstein, 1998). MMR builds the final list one item at a time. At each step it picks the candidate that maximizes

$$\text{MMR}(i) = \lambda \text{Value}(i) - (1 - \lambda) \max_{j \in S} \text{Sim}(i, j)$$

where S is the set of items already selected and Sim is the cosine similarity between item embeddings. With $\lambda = 1$ the list is ordered purely by value; smaller values of λ trade value for variety. Chapter 8 uses audio embeddings from the Yambda dataset, so MMR avoids placing acoustically similar tracks next to each other. The service loads those embeddings at startup, and they are versioned with the model artifacts, separately from the configuration, because a change of embeddings changes what "similar" means.

The cost of MMR depends on how it is implemented. Selecting K items from N candidates by recomputing each candidate's maximum similarity to the whole selected set at every step costs on the order of K^2N similarity computations. Keeping each candidate's running maximum and updating it only against the item selected in the latest step reduces the cost to the order of KN. For a final list of 20 items drawn from 100 candidates, either version is cheap.

13.6.4 Business Rules

The last step applies hard constraints, which Chapter 8 introduced as policy constraints and artist pacing (Sections 8.5.1 and 8.5.3). Typical rules cap the number of tracks per artist or per album in one list, require a minimum gap between tracks by the same artist, reserve particular positions, and exclude

items that must never appear in this context, repeating the eligibility checks from retrieval as a final safeguard. The companion code's re-ranker applies Chapter 8's caps of two tracks per artist and one per album. The rules live in the same versioned configuration as the weights and pass through the same validation.

Rules can conflict with the candidates available. If a cap on tracks per artist leaves fewer eligible items than the client requested, the service needs a defined behavior, either returning a shorter list or relaxing named rules in a stated order, and it should log which happened. Section 13.8.5 covers what happens when the re-ranker itself is unavailable.

13.7 Online Features in the Request Path

Chapter 4 built the feature layer this pipeline reads from: the offline and online stores and batched online lookups (Section 4.2), training-serving skew (Section 4.3), point-in-time correctness (Section 4.4), freshness (Section 4.8), and the silent failures of feature pipelines (Section 4.9). This section adds only what changes when feature lookups sit inside a request with a deadline.

13.7.1 Who Fetches Which Features, and When

The orchestrator owns feature hydration, the step of fetching the feature values a model needs for a request. User features depend only on the request, so the orchestrator fetches them by user identifier at the same moment it calls retrieval, and the two calls overlap. Context features such as the hour of the day or the device type are derived from the request itself. Item features depend on which items retrieval returns, so they cannot be fetched until retrieval finishes. The orchestrator then fetches them for all candidates in a single batched lookup, often called a multi-get, keyed by the candidate identifiers.

One multi-get pays the store's fixed round-trip cost once instead of a thousand times, and passing the values to both rankers keeps the rankers functions of their input, easy to test in isolation.

DoorDash's prediction service, Sibyl, shows the same concerns in a production system. It loads models into memory at startup, fetches the features each prediction needs from a Redis-backed feature store at request time, and falls back to default values configured per model when a feature is unavailable (Zeng, 2020).

13.7.2 Missing and Stale Values

Every lookup can come back incomplete. A new user has no row at all, which sends retrieval down the user cold-start path (Section 13.3.6) and leaves every user feature at its default. A feature pipeline that failed overnight leaves values that exist but are old. The online store usually enforces a time-to-live (TTL), a period after which it treats a stored value as expired and returns nothing for it, and feature platforms typically let you set the TTL per group of features.

The feature contract's maximum staleness (Section 13.2.4) adds a finer check per feature. These checks catch absent and old values, but not a value that is fresh and wrong, such as an engagement aggregate that a broken upstream join replaces with a configured default of zero; Chapter 15 opens with that failure.

A lookup against a networked store also has a timeout derived from the request's deadline (Section 13.8.3); a store that reads local files, such as the companion code's SQLite or Parquet default, is read directly. If the user-feature lookup does not return in time, the orchestrator proceeds with defaults: recommendations become less personal for that request, and the request still meets its deadline. If the

item-feature lookup fails, neither ranker can score, and the request falls back to retrieval's order (rung 4 of Figure 13.6).

13.7.3 The Feature Store Behind One Interface

The companion code hides the feature store behind one small interface: a call that returns a user's features and recent items, and calls that return item features and user-item features for a list of item identifiers. User and item values come with the time they were computed, and user-item values share the user's time; context features, such as the hour, come from the request. The default implementation reads SQLite or Parquet files, so the code runs on a laptop with no extra services. An optional implementation uses Redis, an in-memory key-value store widely used as an online feature store, started through Docker Compose, a tool that starts several containers together from one file. Services depend only on the interface, so switching backends is a configuration change.

Teams that adopt a feature platform can place it behind the same interface. Feast, the open-source feature store discussed in Chapter 4, provides a registry of feature definitions, materialization into online stores such as Redis, and one interface for offline and online access (Feast, 2026). Managed feature stores from cloud providers fill the same role, and Table 13.6 lists the managed in-memory stores that commonly serve as the online layer.

13.8 Orchestration Under a Deadline

With the four stage services defined, the orchestrator ties them into one request, calling the feature store and the stages in the order Figure 13.2 shows. It runs no model, holds no per-user state, and in the companion code is one more FastAPI service.

13.8.1 Why an Orchestrator

A client could in principle call each stage itself, but then every pipeline change would require a client release, and logic that spans stages, such as the deadline, the fallbacks, and per-stage logging, would be scattered across clients written by different teams. The orchestrator keeps all of this in one service that the recommendation team owns and can release independently. It also writes one structured log record per request with the trace identifier, the latency of each stage, the versions each stage reported, and the fallbacks used, the raw material for the latency analysis in Chapter 14 and the monitoring in Chapter 15.

13.8.2 Concurrent Calls with asyncio

The orchestrator spends almost all its time waiting for other services to answer. Python's `asyncio` library supports asynchronous programming, in which a function can start a network call and give up control while it waits, letting the same process work on other requests and other calls in the meantime. Paired with an asynchronous HTTP client such as `httpx`, one orchestrator process can keep many calls in flight.

The payoff is that independent calls overlap. Fetching user features and calling retrieval one after the other costs the sum of their latencies; starting both at once costs only the slower of the two. In code, the orchestrator starts both calls and waits for both results together:

```
import asyncio
```

```
features, retrieval = await asyncio.gather(
    fetch_user_features(request, deadline),
    call_retrieval(request, deadline),
    return_exceptions=True,
)
```

The `return_exceptions=True` argument makes a failure in one call come back as a value instead of cancelling the other, so the orchestrator can apply the fallback for whichever call failed and keep the result of the one that succeeded.

The model services have the opposite profile: they spend their time computing, and a long computation inside an asynchronous function would block every other request in the process. FastAPI handles this by running endpoints declared as ordinary functions, without `async`, in a pool of worker threads (FastAPI, 2026), which is how the ranking services in this chapter declare their scoring endpoints.

13.8.3 Deadline Propagation

A timeout is a duration, such as "wait at most 50 milliseconds for this call". A deadline is a point in time, such as "this request must be answered by 12:00:00.200" (gRPC, 2026a). The orchestrator works with deadlines. When a request arrives, it computes the request's deadline from its arrival time and the latency target, and every downstream call receives, as a duration, the part of the remaining time it may use.

Passing the remaining time down the call chain is called deadline propagation. A deadline is set once at the top of the stack and shared by every call made on behalf of the request, so each service can see how much time is left and give up when the caller has already given up (Beyer et al., 2016). Without propagation, a service deep in the chain can spend its full fixed timeout on a request whose caller stopped waiting long ago, wasting capacity exactly when the system is under strain. Services should receive the remaining time as a duration in a request header, not as an absolute clock time, because the clocks of different machines disagree slightly; gRPC converts deadlines to remaining durations for this reason (gRPC, 2026a).

The timeout for each call is the smaller of a cap for that stage and the time left before the deadline minus a reserve, which covers what must happen after the call, such as a fallback, metadata hydration, and writing the response. This avoids a trap in the illustrative budgets of Figure 13.2. If each stage's timeout were set equal to that stage's p95 latency, about 5% of calls to each stage would time out by design, and the stages together would fail over far more often than any one of them. With derived timeouts, a stage that finishes early leaves its unused time to the stages after it, and a request is cut short only when the deadline itself is at risk. When a timeout fires, the orchestrator cancels the outstanding call so the abandoned work stops consuming a connection.

13.8.4 Retries That Do Not Make Things Worse

Retrying a failed call is tempting, and done carelessly it turns a slow service into a failed one. Brooker (2019) gives the arithmetic: if each of five layers in a call chain makes three attempts, a failure at the bottom can multiply the load on the lowest layer by 243. Retries also arrive exactly when a service is overloaded, which is when extra load hurts most.

The orchestrator follows a few rules. It retries in one place only, itself; the stage services do not retry their own dependencies. It retries at most once, and only for failures a second attempt can plausibly fix, such as a refused connection, never for validation errors (Section 13.2.2). It retries only idempotent

operations, meaning that performing them twice has the same effect as performing them once, which holds for every read-only scoring call in this pipeline. It waits a short, randomized interval first, because retries that fire at the same instant arrive as a synchronized burst; capped exponential backoff with random jitter spreads them out (Brooker, 2019). It retries only if the remaining time allows another attempt. And it limits retries across the whole process to a small budget, so that a widespread failure produces skipped retries and degraded responses instead of a retry storm (Beyer et al., 2016). Section 14.9.2 shows how to choose timeout values from measured percentiles and how hedged requests draw on the same retry budget.

13.8.5 Circuit Breakers and the Degradation Ladder

A service that is failing or timing out on most calls should stop receiving them for a while, both to protect the caller's deadline and to give the service room to recover. A circuit breaker wraps calls to one service and tracks their outcomes (Nygard, 2018; Fowler, 2014). In the closed state, calls pass through normally. When failures exceed a threshold, the breaker opens, and calls fail immediately without contacting the service, so the caller moves straight to its fallback. After a cool-down period, the breaker becomes half-open and lets a trial call through. If the trial succeeds, the breaker closes; if it fails, the breaker opens again for another cool-down. State changes are worth logging, because a breaker that opens is an operational event (Fowler, 2014).

The threshold should be a failure rate over a recent window with a minimum number of calls, such as more than half of at least twenty calls in the last ten seconds, and not a short run of consecutive failures, which normal tail latency can produce by chance. Combined with timeouts derived from the deadline, this keeps the breaker from tripping on ordinary slow requests.

When a breaker is open or a call fails, the orchestrator descends a degradation ladder, shown in Figure 13.6, which defines a fallback for every stage (compare the fail-safes in Section 8.8.3). If the heavy ranker is unavailable, the orchestrator sends the lightweight ranker's scores to the re-ranker, which uses them directly as values when task scores are absent, so the response stays ordered by a real model score. If the re-ranker is unavailable, the orchestrator sorts by the best model score it has and applies a minimal copy of the hard rules, the exclusions and the per-artist cap, so that the fallback never shows content the policy forbids. The lower rungs follow the same pattern down to a cached or precomputed list, so that even a request on which every stage fails receives a useful answer instead of an error. Chapter 14 (Section 14.9.3) adds rungs triggered by load, extends those triggered by time, and measures what each rung costs in quality.

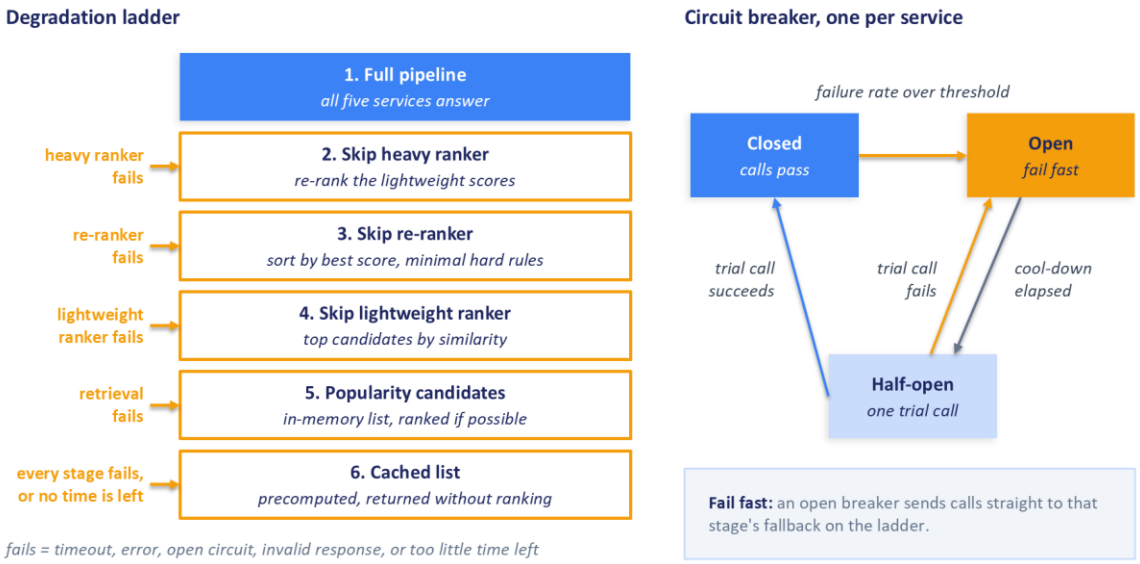


Figure 13.6: Partial failure and the degradation ladder. Each failing stage has a defined fallback (left), and a circuit breaker (right) stops calls to a failing service until a trial call succeeds.

A response reports the lowest rung reached and lists every skipped stage. Missing or stale features use contract defaults and are flagged on the response, not as a rung; if item features are unavailable, neither ranker can score, and the request takes rung 4. A degraded response looks normal to the user, which is the point, and it would hide a failing stage from the team if it went uncounted, so the fallback rate per stage belongs alongside latency and error rate as a basic health signal.

13.8.6 Serving Profiles of the Five Services

The five services differ in what they hold, how often their artifacts change, and how they should scale. Table 13.4 summarizes those profiles, which Section 13.9 maps onto runtimes.

Table 13.4: Serving profiles of the five services.

Service	Compute	Held in memory	Artifacts refreshed	Scaling signal	Fallback when unavailable
Orchestrator	CPU	Popularity list and a minimal copy of the hard rules	With each release	Concurrent requests per replica	Cached or precomputed list (last rung)
Retrieval	CPU	User tower, content-embedding table, ANN index	Index on a schedule or by streaming; the whole unit with each model version	Concurrent requests per replica; memory for large indexes	Popularity list

Service	Compute	Held in memory	Artifacts refreshed	Scaling signal	Fallback when unavailable
Lightweight ranker	CPU	XGBoost model and encoders	With each model release	Concurrent requests per replica	Top retrieval candidates by similarity
Heavy ranker	CPU by default; accelerator after measurement	MMoE weights, feature processor, embedding tables	With each model release	Concurrent requests or latency; queue depth if batching across requests	Lightweight ranker scores
Value function and re-ranker	CPU	Active configuration version and item embeddings for MMR	Configuration on publish, by hot reload; embeddings with their own release	Concurrent requests per replica	Orchestrator sorts by model score and applies minimal hard rules

13.9 Packaging and Infrastructure

This section covers how the services are packaged, where they run, how they reach each other, and how the infrastructure itself is described. Each pattern is cloud-neutral, and Table 13.6 at the end maps the component roles onto the three largest cloud providers.

13.9.1 Containers and Images

A container is an isolated process that carries its own file system, libraries, and configuration while sharing the operating system kernel of the machine it runs on. It starts from an image, a read-only template that holds the base operating system layer, the Python runtime, the libraries, the service code, and optionally the model artifacts. Docker is the most widely used tool for building images and running containers, and a Dockerfile is the text file of instructions from which Docker builds an image, one layer per instruction (Docker, 2026).

A few habits keep images reliable. Put rarely changing layers, such as the base image and the libraries, before frequently changing ones, such as the service code, because Docker reuses unchanged layers from earlier builds. Use multi-stage builds, in which an early stage installs build tools and compiles dependencies and the final stage copies only the results (Docker, 2026). A build can read only files inside its build context, the directory handed to Docker at build time, so a shared package such as the contracts must sit inside it, which usually means building every service from the repository root.

Model artifacts can be baked into the image, which makes the image a complete, self-contained release, or loaded from versioned storage at startup, which keeps images smaller and lets one image serve several model versions selected by a version pointer (Section 13.10.5). An image that includes GPU libraries, such as the CUDA runtime, is larger than a CPU-only image and slower to pull and start. Measure each image's size after building it, and track it like any other number that affects startup time.

13.9.2 Registries, Configuration, and Secrets

Built images are pushed to a container registry, a versioned store of images from which runtimes pull them. Each image has a digest, a hash of its contents that changes whenever anything in the image changes, and one or more tags, human-readable names such as a version number. Tags can be moved to point at a different image, while a digest always identifies exactly one, so releases should refer to images by digest or by tags that the team never reuses.

The same image should run unchanged in every environment, so environment-specific settings, such as service addresses, timeouts, the feature store location, and the active model version, belong in configuration supplied at run time, typically as environment variables (Wiggins, 2017). Secrets, such as database passwords and API keys, live in a secrets manager, a service that stores credentials encrypted and controls who can read them; the runtime injects them at start, and they never appear in an image, a source repository, or a log.

Most teams run at least three environments: development on a laptop; staging, a production-like environment where integration and smoke tests run (Section 13.11); and production. The image digest that passed its tests in staging is the one promoted to production, a principle Humble and Farley (2010) put as building your binaries only once.

13.9.3 Where Containers Run

Two families of runtime cover most needs. A serverless container platform takes an image and a few settings, such as memory, CPU, the number of concurrent requests each replica accepts, and the minimum and maximum number of replicas, and it runs and scales the replicas for you, usually including a public HTTPS endpoint. Cloud Run, Azure Container Apps, and AWS Fargate are examples. These platforms scale on concurrent requests, can scale to zero, and leave little to operate, at the cost of less control over hardware and networking and of the cold starts discussed in Section 13.5.2.

Kubernetes is open source and grew out of Google's experience with its internal cluster managers (Burns et al., 2016). Each machine is a node, and on managed services nodes of the same machine type, such as nodes with accelerators, form a node pool. The smallest unit Kubernetes schedules is a pod, one or more containers that share a network address. A Deployment declares how many replicas of a pod should run and replaces them gradually when the pod's definition changes, and a Service gives a set of pods one stable network address. Managed Kubernetes services run the cluster's control machinery, while your team still owns the workloads, their configuration, and the upgrades.

The serving profiles in Table 13.4 suggest a placement: all five services, while they run on CPU, suit a serverless platform; an accelerated heavy ranker can move to a Kubernetes node pool with accelerators or a serverless platform that offers them; and a stateful sharded embedding lookup belongs on Kubernetes or a dedicated platform. The companion code's deployment script targets Cloud Run because a CPU-only serverless deployment is the smallest step from a laptop to a managed runtime. It sets the minimum number of replicas to zero to keep a test deployment cheap, so its first request after an idle period pays the cold start.

13.9.4 Networking Between Services

Only the gateway and load balancer should be reachable from the public internet. The orchestrator, the stage services, the feature store, and the vector index sit on a private network, which in a public cloud is a virtual private cloud (VPC), an isolated network in which your resources receive private addresses. Serverless services usually run outside your VPC by default and need explicit egress configuration to

reach dependencies with only private addresses, a common reason a service that works locally fails after deployment.

Identity is the second layer of protection. Each cloud has an identity and access management (IAM) system that decides which identities may perform which actions on which resources, and each service should run under its own identity with the least privilege it needs: the re-ranker may read the configuration bucket and nothing else, and only the orchestrator's identity may call the stage services.

13.9.5 REST or gRPC

The services in this chapter talk to each other with REST over HTTP, sending JSON, a text format of nested key-value pairs. REST is easy to debug, because any request can be sent and read by hand with curl, a command-line tool for sending HTTP requests, and FastAPI produces REST endpoints and their documentation directly from the Pydantic contracts.

Many high-throughput systems use gRPC for internal calls instead. gRPC is an open-source framework for remote procedure calls in which a client calls a method on a server on another machine as if it were a local function (gRPC, 2026b). Services and messages are defined in Protocol Buffers, a language-neutral format in which each message is declared in a `.proto` file with a numbered field for every value. A compiler generates client and server code in many languages from that file, and messages travel in a compact binary encoding. The field numbers make schema evolution explicit: old code ignores fields it does not know, and a deleted field's number must be reserved and never reused (Protocol Buffers, 2026). DoorDash's Sibyl prediction service receives requests over gRPC with Protocol Buffers (Zeng, 2020). Table 13.5 compares the two.

Table 13.5: REST with JSON compared with gRPC with Protocol Buffers.

Consideration	REST with JSON	gRPC with Protocol Buffers
Payload	Human-readable text	Compact binary encoding
Interface definition	Optional; in this chapter, Pydantic classes and generated OpenAPI documents	Required `.proto` files from which code is generated
Debugging	Send and read requests by hand with curl or a browser	Needs gRPC-aware tools to inspect traffic
Parsing cost	Higher, especially for large payloads such as a thousand candidates with features	Lower
Deadlines	Implemented by the application, for example with a header	Built into the framework, with propagation to downstream calls
Schema evolution	Rules enforced by your validation code and review	Field numbers and reserved fields in the schema
Fits	Client-facing APIs, small teams, early systems; the companion code	High-volume internal calls between services owned by one organization

The protocol leaves the contracts, the deadline, and the fallbacks unchanged. Chapter 14 attributes serialization time as its own stage and lists binary encoding among its levers (Section 14.4.4).

13.9.6 Infrastructure as Code

Running commands by hand is adequate for a first deployment, but then the environment exists only because someone ran the right commands in the right order, and recreating it after a disaster is manual and error-prone.

Infrastructure as code (IaC) describes the resources, such as services, networks, identities, and storage, in configuration files kept in version control next to the application code. Terraform is a widely used IaC tool: you write declarative configuration that describes the desired end state, Terraform produces a plan showing what it will create, change, or destroy, and on approval it applies the plan through the providers' APIs (HashiCorp, 2026). Infrastructure changes then go through review like code, and every environment can be rebuilt from the same files. The companion code's Cloud Run deployment script is a smaller step in the same direction, recording the deployment steps in a file instead of in someone's memory.

13.9.7 Mapping the Topology to a Provider

Figure 13.7 shows the whole deployment in cloud-neutral terms: five replicated services behind the edge, and the shared components they depend on.

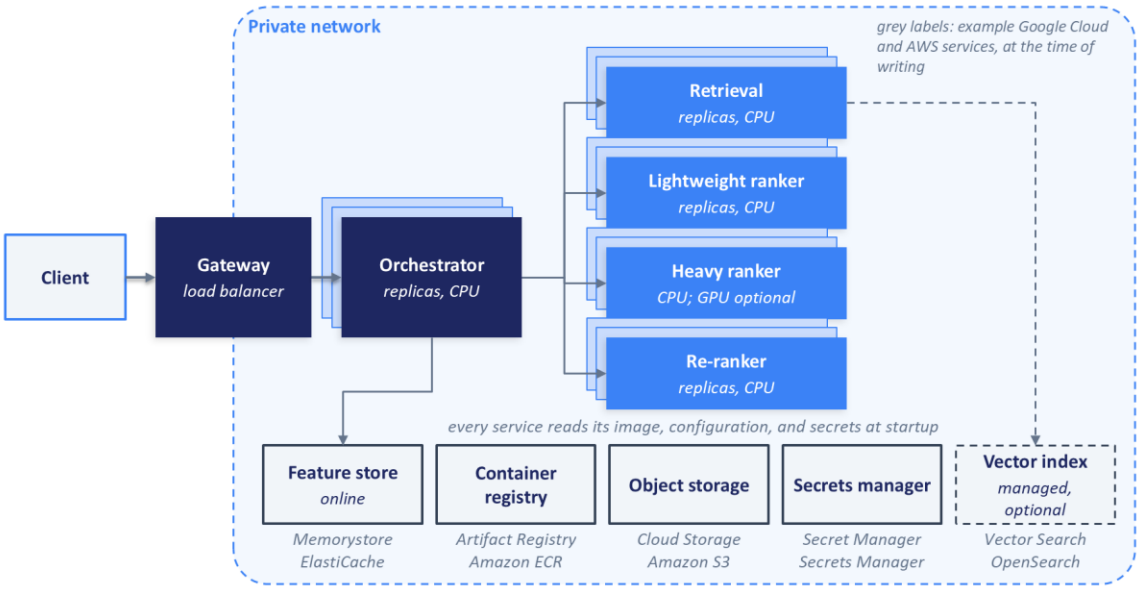


Figure 13.7: A cloud-neutral deployment topology. Only the edge is public; the five services and the shared components they depend on sit inside a private network.

Table 13.6 maps each component role to an example service from Google Cloud, Amazon Web Services, and Microsoft Azure. Product names change often: Google's managed vector search service, formerly Vertex AI Vector Search, is now Vector Search on Gemini Enterprise Agent Platform (Google Cloud, 2026d), and Microsoft is retiring Azure Cache for Redis in favor of Azure Managed Redis (Microsoft, 2026). Treat the table as a starting point for your own search, accurate at the time of writing. Prices

change more often than names; check each provider's pricing page, such as Cloud Run's (Google Cloud, 2026e), before sizing a deployment.

Table 13.6: Component roles with example provider services, at the time of writing.

Component role	Google Cloud	Amazon Web Services	Microsoft Azure
Serverless containers	Cloud Run	AWS Fargate with Amazon ECS	Azure Container Apps
Managed Kubernetes	Google Kubernetes Engine (GKE)	Amazon EKS	Azure Kubernetes Service (AKS)
Managed vector search	Vector Search on Gemini Enterprise Agent Platform (formerly Vertex AI Vector Search)	Amazon OpenSearch Service with vector search	Azure AI Search
In-memory store for online features	Memorystore for Redis	Amazon ElastiCache	Azure Managed Redis
Object storage for configurations and artifacts	Cloud Storage	Amazon S3	Azure Blob Storage
Container registry	Artifact Registry	Amazon ECR	Azure Container Registry
Secrets manager	Secret Manager	AWS Secrets Manager	Azure Key Vault
Load balancing and API gateway	Cloud Load Balancing and API Gateway	Elastic Load Balancing and Amazon API Gateway	Azure Load Balancer, Application Gateway, and Azure API Management

13.10 Releasing Safely

A recommender changes constantly: code changes, models are retrained, indexes are rebuilt, and value-function configurations are revised, each on its own schedule. This section covers the general mechanics of releasing a change safely; the criteria that decide whether a given release may proceed belong to Chapter 15.

13.10.1 Immutable, Versioned Artifacts

Every unit that can be released, whether a container image, a model artifact, an index, or a configuration, receives a unique version when it is published and is never modified afterward. Releasing

then means changing which version serves traffic, and rolling back means changing it back; a file overwritten in place allows neither.

A model registry adds stages: a newly trained model enters as a candidate and is promoted to staging and then to production as it passes each check. Promotion moves a label or pointer and never edits the artifact, so the version that was tested is exactly the version that serves. Breck et al. (2017) list the machine learning counterparts among their production-readiness tests: a model should go through a canary process before it serves all traffic, and it should be possible to roll it back to the previous serving version quickly and safely.

13.10.2 Health, Readiness, and Liveness

A runtime needs to know whether each replica can do its job, and services answer through health checks, small endpoints that the platform calls on a schedule. Kubernetes distinguishes three kinds, and serverless platforms offer similar ones (Kubernetes, 2026a).

A liveness check asks whether the process is still working at all; if it fails repeatedly, the platform restarts the container, which recovers a replica that has deadlocked or exhausted its memory. A readiness check asks whether the replica can serve requests right now; if it fails, the platform stops sending traffic to that replica without restarting it, and it keeps checking, so a replica can leave and rejoin the pool. A startup check protects slow starters: until it succeeds, the platform does not run the other checks, so a replica that needs time to load a model is not killed for being slow to answer (Kubernetes, 2026a).

Readiness should turn true only after every artifact has loaded and passed its startup checks: the manifest agrees with the loaded tower and index, the item tower reproduces a stored sample of index vectors, and the feature contract matches the model. Liveness should check only the process itself. If liveness depended on the feature store, a feature store outage would make the platform restart every replica at once, turning a partial failure into a total one. The same caution applies, more mildly, to readiness: a service that reports itself unready whenever a dependency is slow removes capacity exactly when the degradation ladder could have kept it serving. Replicas also need to stop gracefully: on the termination signal that the platform sends during a release, the service should stop accepting new requests, finish those in flight, and then exit.

13.10.3 Shifting Traffic: Rolling, Blue-Green, and Canary

Three common ways to move traffic to a new version trade speed against caution.

A rolling update replaces replicas a few at a time: new replicas start, become ready, and receive traffic, while old replicas drain and stop. It is the default for a Kubernetes Deployment, which lets you set how many extra replicas may be created and how many may be unavailable during the update (Kubernetes, 2026b). It needs little spare capacity, and for a while old and new versions serve traffic side by side, which is why contracts must stay compatible across a release (Section 13.2.3).

A blue-green deployment keeps two complete environments. The new version is deployed and tested in the idle one, and the router switches all traffic to it at once; rolling back means switching the router back (Fowler, 2010). The atomic switch suits changes that must not mix, such as a retrieval unit whose tower and index must change together, at the cost of running two full environments during the switch.

A canary release, also called a phased or incremental rollout, sends a small share of traffic to the new version first, compares its behavior with the stable version, and increases the share in steps only while it looks healthy (Sato, 2014). Cloud Run supports this directly: a new revision is deployed with no traffic, then given a percentage, such as 5%, and raised as its performance proves satisfactory (Google Cloud,

2026b). Michelangelo, Uber's machine learning platform, supports gradual switching of traffic between model versions in a similar way (Del Balso and Hermann, 2017). For recommendation traffic, the canary share should be assigned by a hash of the user identifier, so that each user sees a consistent version. Platform traffic splits assign requests, not users, so a user-consistent canary needs the assignment in the gateway or orchestrator.

Sato (2014) warns against conflating a canary with an A/B test. A canary asks whether a release is safe to complete, judged mostly on errors, latency, and guardrail metrics over hours; an A/B test asks whether a change improves outcomes, which requires a planned experiment with adequate statistical power, the subject of Chapter 16. Many teams also run a shadow deployment before a canary: the new version receives a copy of live requests and its responses are recorded and discarded, so its latency and outputs can be checked without affecting users. DoorDash's Sibyl supports shadow predictions of this kind (Zeng, 2020).

The strategies combine naturally: routine code changes to stateless services can roll, model changes deserve a shadow period and a canary, and a retrieval unit is switched as a whole, blue-green style, inside each replica through the version pointer described below.

13.10.4 Rollback

A rollback is only as fast as its preparation. The previous version should remain deployed and warm during the rollout and for a period afterward, so that rolling back is a traffic shift or a pointer change and not a rebuild. Kubernetes keeps the previous revisions of a Deployment and can return to one with a single command (Kubernetes, 2026b), and Cloud Run can route traffic back to an earlier revision in the same way it routes traffic forward (Google Cloud, 2026b).

Rollback also depends on compatibility: if the new version wrote data in a format the old version cannot read, rolling back breaks it, which is another reason to split breaking contract changes into separate releases. The conditions that trigger a rollback, such as a burst of contract rejections, a rise in errors or fallbacks, or a latency regression, should be defined before the release starts; Chapter 15 discusses how to choose and judge them. A rollback procedure that has never been exercised may fail when it is needed most, so it should be rehearsed in staging.

13.10.5 Version Pointers for Models and Indexes

Models and indexes change more often than code, and redeploying a service to change its model couples two unrelated releases. A version pointer decouples them. The service code is deployed once, and a small record in the model registry or in configuration names the artifact version each service should serve, such as retrieval unit 42 and heavy-ranker model 17. When its pointer changes, a service loads the new version in the background, runs its startup checks against it, and switches a single in-memory reference to it, keeping the old one loaded for a quick revert. Requests in flight finish on the version they started with, so no request sees a mixture. The companion code takes the simpler route for models and indexes: each service reads the pointer at startup, so a change takes effect when the services restart, or as a new revision on Cloud Run. Only the re-ranker's configuration switches in memory (Section 13.6.2), and on Cloud Run the companion code bakes the configuration into the image, so there a configuration change is also a new revision.

Michelangelo uses the same idea: deploying a new model under an existing tag switches that tag's traffic immediately, and a serving container can hold several models at once so that transitions are safe (Del Balso and Hermann, 2017). For retrieval, the pointer names the whole unit of Section 13.3.4. Chapter 15

covers the compatibility windows and release checks needed to coordinate model, embedding, and index releases (Section 15.9.4).

13.10.6 The Release Pipeline End to End

Figure 13.8 assembles these pieces into one pipeline. A commit triggers CI, which builds the image and runs the unit and contract tests, so a breaking schema change fails before it reaches any running service. After staging runs the integration and smoke tests of Section 13.11, the release proceeds from a small canary share to all traffic.

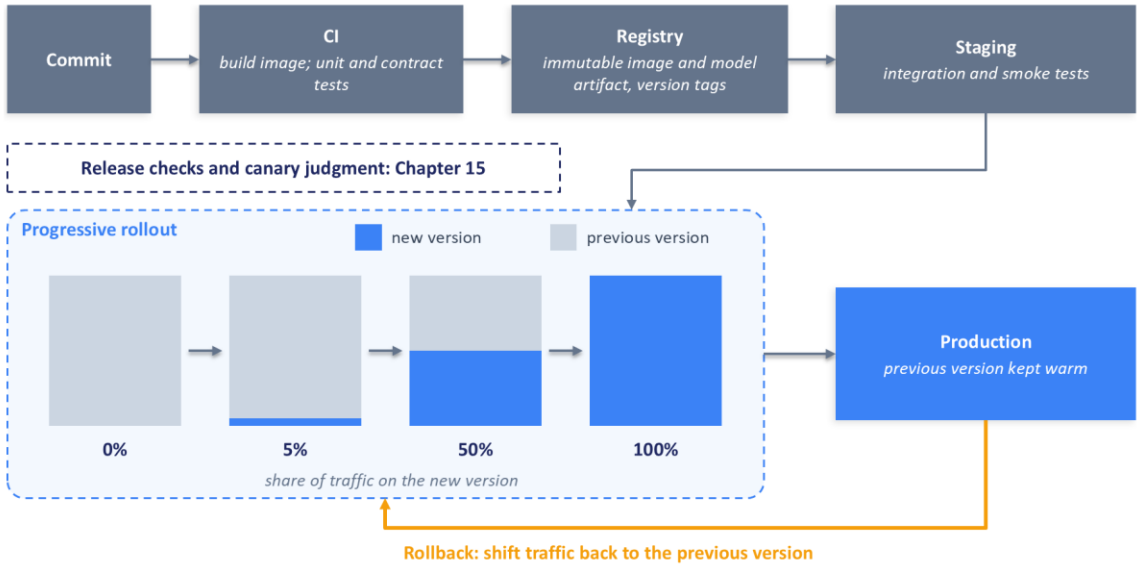


Figure 13.8: Build, promote, and roll out. Immutable artifacts move from CI through staging to a progressive rollout, and traffic can return to the previous version at any step. The checks that decide each step are covered in Chapter 15.

Configuration changes to the value function follow the same path in a lighter form (Section 13.6.2). The checks that decide whether each step may proceed, such as which metrics a canary must hold, for how long, and how the comparison is judged, are the subject of Chapter 15 (Section 15.9.5).

13.11 Validating the Deployed Pipeline

A pipeline that starts is not yet a pipeline that works. Validation proceeds in layers, from one request sent by hand to a load test, and Table 13.7 summarizes them.

13.11.1 A Single Request by Hand

The first check is one request sent by hand, first to each service in isolation and then to the orchestrator. With the services running locally, a request to the orchestrator looks like this, with the address adjusted to the one the companion code's README gives:

```
curl -s -X POST http://127.0.0.1:8000/v1/recommend \
  -H "Content-Type: application/json" -H "X-Request-Timeout-Ms: 1000" \
  -d '{"user_id": 10300, "num_results": 10}'
```

User identifiers that exist in the deployed data are listed in the bundle's `sample_users.json`.

Quoting rules differ between shells, and on Windows PowerShell's `Invoke-RestMethod` command is a convenient alternative. Read the response with the pipeline in mind. It should validate against the client contract and carry a trace identifier, the versions of every artifact that served it, and the rung of the degradation ladder it reached, which shows whether the candidates came from ANN search or from the popularity list. Searching the services' logs for the trace identifier shows which stages the request reached, how long each took, and which versions answered.

Two deliberate mistakes complete the manual check. Send a malformed request, such as one with a negative `num_results`, and confirm that the orchestrator returns a 422 response that names the field. Then stop one stage service, resend the original request, and confirm that the response still arrives within the deadline, marked as degraded at the expected rung of the ladder. A fallback that has never been triggered on purpose is untested code.

13.11.2 Contract, Integration, and Smoke Tests

Manual checks do not scale, and they do not run on every change. The companion code automates the next layers with pytest, a Python testing framework in which tests are ordinary functions that make assertions.

Contract tests validate example payloads against the shared Pydantic contracts and call each service's endpoint inside the test process, without a network, checking that its responses validate; they are the cheapest place to catch a field renamed on one side of a boundary only. Integration tests run all five services together, in the companion code inside one test process with failures injected at each service boundary, and assert on end-to-end results: the number of items, the absence of excluded items, the trace and version fields, and the fallback taken when a stage fails or stops answering. Smoke tests are a handful of fast requests after each deployment, a health check per service and one recommendation request per main path.

Table 13.7: Validation activities for the deployed pipeline.

Activity	Purpose	When it runs	What it catches
Manual request	See one request end to end, with its trace and versions	During development and after major changes	Obvious wiring and configuration mistakes
Unit tests	Check individual functions, such as the value function and MMR	On every change, in CI	Logic errors inside one service
Contract tests	Check each service against the shared request and response contracts	On every change, in CI	Renamed, missing, or retyped fields at a boundary
Integration tests	Run the whole pipeline, including deliberate stage failures	Before promoting a release, locally or in staging	Mismatched assumptions between services; broken fallbacks

Activity	Purpose	When it runs	What it catches
Smoke tests	Confirm a fresh deployment is alive and connected	Immediately after each deployment	Missing secrets, unreachable dependencies, failed startup checks
Load tests	Observe latency, throughput, and errors as load rises	Before launch and after significant changes, in a production-like environment	Capacity limits, tail-latency growth, saturation behavior

13.11.3 Load Testing with Locust

Locust is an open-source load-testing tool in which test scenarios are written as plain Python. Each simulated user runs as a lightweight coroutine, so one process can simulate many concurrent users, and Locust reports response times, throughput, and errors as the test runs, either in a web interface or from the command line (Locust, 2026).

The companion code's Locust file defines a simulated user that sends a recommendation request, waits for the response, and sends the next one at once, so each simulated user is one request in flight. A useful first test ramps the number of simulated users in steps, holding each step for long enough to read stable numbers, and records the p50, p95, and p99 latency, the throughput, the error rate, and the fallback rate at each step. The step at which latency starts to climb steeply, or fallbacks start to appear, marks the practical capacity of the deployment under test.

Two cautions apply. First, a load test measures the environment it runs in: a run on a laptop exercises the code paths and finds gross inefficiencies, but capacity tests belong in an environment that resembles production. Second, this is a closed loop, in which offered load drops when the system slows down, so it can understate tail latency under overload; Section 14.3.3 explains the effect and how to measure around it.

The Locust file is a starting point, and this chapter reports no measurements from it, because the numbers depend entirely on the machine and the deployment. Chapter 14 starts from load tests of this kind, measures where the time goes, and applies the optimizations this chapter deferred.

13.12 Summary

This chapter turned the models from Chapters 5 to 8 into a pipeline of five services: retrieval with the two-tower model, lightweight ranking with XGBoost, heavy ranking with the MMoE, the value function and re-ranker, and an orchestrator that ties them together behind one endpoint. The work divides into what happens ahead of time, which produces versioned artifacts, and what happens while a user waits, which must finish within a deadline.

Four patterns carry the design: typed contracts that make every boundary explicit and every request traceable; a deadline propagated to every call; a ladder of fallbacks, guarded by circuit breakers, that gives every request a useful answer; and immutable, versioned releases with health checks, progressive traffic shifting, and version pointers, so that any change can be undone. A layered set of tests, from one manual request to a Locust load test, validates the result.

What We Deferred

Two topics remain for the next chapters, and a third follows them. Chapter 14 (Latency Optimization) starts from load tests like the one in Section 13.11 and makes the pipeline faster, through request batching, caching, model distillation and quantization, and infrastructure tuning, with budgets derived from measurement in place of the illustrative split in Figure 13.2. Chapter 15 (Monitoring, Reliability, and Retraining) addresses what happens after deployment: detecting failures that keep the shape of the data while changing its meaning, diagnosing and remediating incidents, deciding when to retrain, and the release checks that decide whether a canary may proceed. Chapter 16 (Experimentation and Bandits) then addresses how to tell whether a change shipped through this pipeline improves outcomes, using controlled experiments, interleaving, and exploration policies.

References

Bernardi, L., Mavridis, T., and Estevez, P. (2019). "150 Successful Machine Learning Models: 6 Lessons Learned at Booking.com." Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 1743-1751. ACM. https://doi.org/10.1145/3292500.3330744

Beyer, B., Jones, C., Petoff, J., and Murphy, N. R. (Eds.). (2016). Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media. https://sre.google/sre-book/table-of-contents/

Breck, E., Cai, S., Nielsen, E., Salib, M., and Sculley, D. (2017). "The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction." 2017 IEEE International Conference on Big Data. IEEE. https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/

Brooker, M. (2019). "Timeouts, Retries, and Backoff with Jitter." Amazon Builders' Library. https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. (2016). "Borg, Omega, and Kubernetes." ACM Queue, 14(1), 70-93. https://doi.org/10.1145/2898442.2898444

Carbonell, J., and Goldstein, J. (1998). "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries." Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, 335-336. ACM. https://doi.org/10.1145/290941.291025

Chen, T., and Guestrin, C. (2016). "XGBoost: A Scalable Tree Boosting System." Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794. ACM. https://doi.org/10.1145/2939672.2939785

Covington, P., Adams, J., and Sargin, E. (2016). "Deep Neural Networks for YouTube Recommendations." Proceedings of the 10th ACM Conference on Recommender Systems, 191-198. ACM. https://doi.org/10.1145/2959100.2959190

Del Balso, M., and Hermann, J. (2017). "Meet Michelangelo: Uber's Machine Learning Platform." Uber Engineering. https://www.uber.com/us/en/blog/michelangelo-machine-learning-platform/

Docker. (2026). "Multi-Stage Builds." Docker Documentation. https://docs.docker.com/build/building/multi-stage/

Faiss. (2026a). "Special Operations on Indexes." Faiss Wiki. https://github.com/facebookresearch/faiss/wiki/Special-operations-on-indexes

Faiss. (2026b). "Guidelines to Choose an Index." Faiss Wiki. https://github.com/facebookresearch/faiss/wiki/Guidelines-to-choose-an-index

Faiss. (2026c). "Setting Search Parameters for One Query." Faiss Wiki. https://github.com/facebookresearch/faiss/wiki/Setting-search-parameters-for-one-query

FastAPI. (2026). FastAPI Documentation. https://fastapi.tiangolo.com/

Feast. (2026). Feast Documentation. https://docs.feast.dev/

Fowler, M. (2010). "BlueGreenDeployment." martinfowler.com. https://martinfowler.com/bliki/BlueGreenDeployment.html

Fowler, M. (2014). "CircuitBreaker." martinfowler.com. https://martinfowler.com/bliki/CircuitBreaker.html

Google Cloud. (2026a). "GPU Support for Services." Cloud Run Documentation. https://cloud.google.com/run/docs/configuring/services/gpu

Google Cloud. (2026b). "Rollbacks, Gradual Rollouts, and Traffic Migration." Cloud Run Documentation. https://cloud.google.com/run/docs/rollouts-rollbacks-traffic-migration

Google Cloud. (2026c). "Best Practices for Autoscaling Large Language Model (LLM) Inference Workloads with GPUs on Google Kubernetes Engine (GKE)." Google Kubernetes Engine Documentation. https://docs.cloud.google.com/kubernetes-engine/docs/best-practices/machine-learning/inference/autoscaling

Google Cloud. (2026d). "Gemini Enterprise Agent Platform Name Changes." Google Cloud Documentation. https://docs.cloud.google.com/gemini-enterprise-agent-platform/vertex-ai-name-changes

Google Cloud. (2026e). "Cloud Run Pricing." Google Cloud. Accessed at the time of writing. https://cloud.google.com/run/pricing

gRPC. (2026a). "Deadlines." gRPC Documentation. https://grpc.io/docs/guides/deadlines/

gRPC. (2026b). "Introduction to gRPC." gRPC Documentation. https://grpc.io/docs/what-is-grpc/introduction/

HashiCorp. (2026). "What Is Terraform?" Terraform Documentation. https://developer.hashicorp.com/terraform/intro

Humble, J., and Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley.

Ivchenko, D., Van Der Staay, D., Taylor, C., Liu, X., Feng, W., Kindi, R., Sudarshan, A., and Sefati, S. (2022). "TorchRec: A PyTorch Domain Library for Recommendation Systems." Proceedings of the 16th ACM Conference on Recommender Systems, 482-483. ACM. https://doi.org/10.1145/3523227.3547387

Johnson, J., Douze, M., and Jégou, H. (2019). "Billion-Scale Similarity Search with GPUs." IEEE Transactions on Big Data, 7(3), 535-547. https://doi.org/10.1109/TBDATA.2019.2921572

Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.

Kubernetes. (2026a). "Configure Liveness, Readiness and Startup Probes." Kubernetes Documentation. https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

Kubernetes. (2026b). "Deployments." Kubernetes Documentation. https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

Kubernetes. (2026c). "Horizontal Pod Autoscaling." Kubernetes Documentation. https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

Li, M., Andersen, D. G., Park, J. W., Smola, A. J., Ahmed, A., Josifovski, V., Long, J., Shekita, E. J., and Su, B.-Y. (2014). "Scaling Distributed Machine Learning with the Parameter Server." 11th USENIX Symposium on Operating Systems Design and Implementation, 583-598. USENIX Association. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/li_mu

Locust. (2026). Locust Documentation. https://docs.locust.io/

Ma, J., Zhao, Z., Yi, X., Chen, J., Hong, L., and Chi, E. H. (2018). "Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts." Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 1930-1939. ACM. https://doi.org/10.1145/3219819.3220007

Microsoft. (2026). "Azure Cache for Redis Retirement FAQ." Microsoft Learn. https://learn.microsoft.com/en-us/azure/azure-cache-for-redis/retirement-faq

Naumov, M., Mudigere, D., Shi, H.-J. M., Huang, J., Sundaraman, N., Park, J., Wang, X., Gupta, U., Wu, C.-J., Azzolini, A. G., Dzhulgakov, D., Mallevich, A., Cherniavskii, I., Lu, Y., Krishnamoorthi, R., Yu, A., Kondratenko, V., Pereira, S., Chen, X., Chen, W., Rao, V., Jia, B., Xiong, L., and Smelyanskiy, M. (2019). "Deep Learning Recommendation Model for Personalization and Recommendation Systems." arXiv:1906.00091. https://arxiv.org/abs/1906.00091

NVIDIA. (2026). "NVIDIA Dynamo-Triton." NVIDIA Developer. https://developer.nvidia.com/dynamo-triton

Nygard, M. T. (2018). Release It! Design and Deploy Production-Ready Software (2nd ed.). Pragmatic Bookshelf.

Protocol Buffers. (2026). "Overview." Protocol Buffers Documentation. https://protobuf.dev/overview/

Pydantic. (2026). Pydantic Documentation. https://docs.pydantic.dev/latest/

PyTorch. (2026). "TorchServe." GitHub repository. https://github.com/pytorch/serve

Qdrant. (2026). Qdrant Documentation. https://qdrant.tech/documentation/

Ray. (2026). "Ray Serve: Scalable and Programmable Serving." Ray Documentation. https://docs.ray.io/en/latest/serve/index.html

Sato, D. (2014). "CanaryRelease." martinfowler.com. https://martinfowler.com/bliki/CanaryRelease.html

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). "Hidden Technical Debt in Machine Learning Systems." Advances in Neural Information Processing Systems 28. Curran Associates. https://papers.neurips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems

Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., and Shanbhag, C. (2010). "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure." Google Technical Report dapper-2010-1. https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/

Uvicorn. (2026). Uvicorn Documentation. https://uvicorn.dev/

Vorotilov, V., and Shugaepov, I. (2023). "Scaling the Instagram Explore Recommendations System." Engineering at Meta. https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/

Wiggins, A. (2017). "The Twelve-Factor App: III. Config." https://12factor.net/config

Zeng, C. (2020). "Meet Sibyl: DoorDash's New Prediction Service." DoorDash Engineering. https://careersatdoordash.com/blog/doordashs-new-prediction-service/